# Sprint 14 Structured Notes: Accessible Dynamic JavaScript

## Section 0: What Sprint 14 Is About (Sprint 14 කියන්නේ මොකක් ගැනද?)

**Concept Explanation (Sinhala):** Sprint 14 එකෙන් ප්‍රධාන වශයෙන්ම කතා කරන්නේ අපේ interactive web pages හැමෝටම පාවිච්චි කරන්න පුළුවන් විදිහට (accessible) හදන්නේ කොහොමද කියන එක ගැනයි.

මීට කලින් sprints වලදී ඔයා ඉගෙන ගත්තා:

- JavaScript වලින් HTML elements හොයාගන්න (find කරන්න) හැටි.
- Page එකේ text සහ content වෙනස් කරන්න.
- Elements අලුතින් එකතු කරන්න සහ අයින් කරන්න (add/remove කරන්න).
- Click, input, form submission වගේ events වලට listen කරන්න.
- Menus open කරන්න, messages update කරන්න, සහ visual state එක වෙනස් කරන්න.

Sprint 14 වලින් ඔයාට professional web developer කෙනෙක් වෙන්න අත්‍යවශ්‍ය පුරුද්දක් කියලා දෙනවා: **JavaScript වලින් user ට පේන visual state එක වෙනස් කරද්දීම, assistive technology වලට තේරෙන state එකත් update වෙන්න ඕනේ.**

Assistive technology කියන්නේ screen readers, keyboard navigation, voice control වගේ, ශාරීරික අපහසුතා තියෙන අයට web sites පාවිච්චි කරන්න උදව් වෙන tools වලටයි.

Sprint 14 වල ප්‍රධානම goal (ඉලක්කය) තමයි: **Visual state එක සහ assistive-technology state එක හැමවෙලේම sync එකේ (එක සමානව) තියාගන්න එක.**

Sprint 14 ඉවර වෙද්දී ඔයාට මේ දේවල් ගැන හොඳ අවබෝධයක් ලැබෙනවා:

- accessible dynamic content
- visual state
- accessibility state
- semantic state
- native HTML first
- ARIA basics (aria-expanded, aria-controls, aria-owns, aria-live, aria-selected, aria-disabled, aria-haspopup, aria-required, aria-checked)
- hidden attribute එක
- custom controls
- keyboard access
- accessible dropdowns
- accessible status messages
- accessible tabs
- පොදුවේ සිදුවන වැරදි (common Sprint 14 mistakes)
- අපේ JavaScript state එක accessible ද කියලා check කරගන්නේ කොහොමද කියන එක.

## Part A: Current Progress Before Sprint 14 (Sprint 14 ට කලින් ඔයා ලබාගත් ප්‍රගතිය)

**Concept Explanation (Sinhala):** Sprint 14 සාර්ථකව කරන්න නම් කලින් sprints වල ඉගෙන ගත්ත දේවල් ගොඩක් වැදගත් වෙනවා. අපි බලමු ඒ දේවල් මේකට උදව් වෙන්නේ කොහොමද කියලා:

| Earlier Sprint | What You Learned | How It Helps in Sprint 14 |
|---|---|---|
| **Sprint 1** | JavaScript වලින් page එකක් respond කරවන්න පුළුවන් බව. | Accessible pages වලටත් හරියටම respond කරන්න මේක උදව් වෙනවා. |
| **Sprint 2** | Values, variables, types, let සහ const ගැන. | ARIA states සාමාන්‍යයෙන් store කරන්නේ සහ update කරන්නේ `"true"` සහ `"false"` වගේ string values වලින්. |
| **Sprint 3** | Strings සහ text processing. | ARIA attributes සහ status messages සාමාන්‍යයෙන් text strings වෙනවා. |
| **Sprint 4** | Booleans, conditions, සහ logical decisions. | දෙයක් open ද, closed ද, selected ද, නැත්නම් disabled ද කියලා තීරණය කරන්න booleans පාවිච්චි වෙනවා. |
| **Sprint 5 & 6** | Functions, scope සහ Arrays ගැන. | Accessible behavior එක සාමාන්‍යයෙන් ලියන්නේ event handler functions ඇතුළේ. ඒ වගේම tabs, menus සහ lists වල එකිනෙකට සම්බන්ධ elements ගොඩක් තියෙන්න පුළුවන් නිසා arrays උදව් වෙනවා. |
| **Sprint 7** | Loops ගැන. | Tabs හෝ buttons හරහා loop කරලා ඒවායේ states update කරන්න loops ඕන වෙනවා. |
| **Sprint 8** | Objects සහ properties. | DOM elements වල අපිට කියවන්න (read) සහ update කරන්න පුළුවන් properties සහ attributes තියෙනවා. |
| **Sprint 9** | Callbacks සහ array methods. | Event listeners වලදී callback functions පාවිච්චි වෙනවා. |
| **Sprint 10** | Sets සහ Maps. | Data organize කරන්න JavaScript වල තියෙන වෙනත් මෙවලම් ගැන අවබෝධයක් ලැබෙනවා. |
| **Sprint 11** | Browser APIs, DOM, සහ element selection. | Accessibility updates කරන්න ඕනේ elements හොයාගන්න මේ දැනුම උදව් වෙනවා. |
| **Sprint 12** | DOM content හදන්න (create), edit කරන්න සහ remove කරන්න. | JavaScript වලින් ආරක්ෂිතව page content එක වෙනස් කරන්න මේකෙන් පුළුවන්. |
| **Sprint 13** | Events, timing, dialogs, සහ event flow. | User, page එකත් එක්ක interact වෙද්දී accessibility state එක update කරන්න මේක උදව් වෙනවා. |

**සරල සාරාංශය (Simple summary):** Sprints 11-13 වලින් ඔයා ඉගෙන ගත්තේ page එකේ elements හොයන්නේ, වෙනස් කරන්නේ සහ ඒවට react කරන්නේ කොහොමද කියලයි. Sprint 14 වලින් ඉගෙන ගන්නේ අන්න ඒ කරන වෙනස්කම් හැම user කෙනෙකුටම එක වගේ තේරෙන විදිහට කරන්නේ කොහොමද කියලයි.

## Part B: Visual State vs Accessibility State (දෘශ්‍ය තත්ත්වය සහ ප්‍රවේශ්‍ය තත්ත්වය)

**Concept Explanation (Sinhala):** JavaScript වලින් page එකක් වෙනස් කරද්දී ප්‍රධාන වශයෙන්ම states වර්ග දෙකක් වෙනස් වෙනවා:

1. **Visual State (පෙනෙන තත්ත්වය):** මේකෙන් අදහස් කරන්නේ user ට ඇස් දෙකෙන් පේන දේ. උදාහරණ:
   - Menu එකක් open වෙලා තියෙනවා.
   - Tab එකක් select වෙලා වගේ පේනවා.
   - Button එකක් disabled වෙලා වගේ පේනවා (හොඳට පේන්නේ නැති විදිහට/fade වෙලා).
   - Screen එකේ අලුත් message එකක් පේන්න ගන්නවා.
   - Section එකක් hide වෙලා තියෙනවා.
2. **Accessibility State (ප්‍රවේශ්‍යතා තත්ත්වය):** මේකෙන් අදහස් කරන්නේ screen readers වගේ assistive technology වලට තේරෙන දේ. උදාහරණ:
   - Screen reader එක දන්නවා menu එකක් expanded (දිගහැරිලා) තියෙනවා කියලා.
   - Screen reader එක දන්නවා select වෙලා තියෙන්නේ මොන tab එකද කියලා.
   - Screen reader එක දන්නවා අපි හදපු custom control එකක් disabled වෙලා තියෙනවා කියලා.
   - Screen reader එකකින්, වෙනස් වුණු අලුත් status message එකක් ශබ්ද නඟලා කියවනවා (announce කරනවා).

**ප්‍රධාන ගැටලුව (The Main Problem):** ඇස් පෙනෙන user කෙනෙකුට page එකේ වෙනස් වෙන දේ පෙනුනත්, assistive technology එකකට ඒ බව දැනගන්න ලැබෙන්නේ නැහැ ඔයා හරියටම HTML හෝ ARIA state එක update කළේ නැත්නම්.

උදාහරණයක් බලමු: Menu එකක් visually open වෙනවා. **නරක කේතය (Bad code):**

```
menu.hidden = false;
```

**වඩා හොඳ කේතය (Better code):**

```
menu.hidden = false;
button.setAttribute("aria-expanded", "true");
```

**Code Breakdown (Sinhala):** පළමු කේතයේ (Bad code) `menu.hidden = false` මඟින් menu එක visually පෙනුනත්, button එක දැන් expanded වෙලා තියෙන්නේ කියලා screen reader එකට දැනගන්න ලැබෙන්නේ නැහැ. නමුත් දෙවන කේතයේ (Better code) එක පාර වැඩ දෙකක් වෙනවා:

1. `menu.hidden = false` මඟින් menu එක screen එකේ පේන්න ගන්නවා (Visual state).
2. `button.setAttribute("aria-expanded", "true")` මඟින් button එක තමන් දැන් "expanded" (දිගහැරිලා) ඉන්නේ කියලා screen reader එකට report කරනවා (Accessibility state).

**ආරම්භක රීතිය (Beginner rule):** JavaScript වලින් UI එක වෙනස් කරද්දී, හැමවෙලේම DOM state එක සහ ARIA state එක එකටම update කරන්න.

## Part C: Accessibility and Dynamic Content (ප්‍රවේශ්‍යතාව සහ ගතික අන්තර්ගතය)

### What Is Dynamic Content?

**Concept Explanation (Sinhala):** Dynamic content කියන්නේ page එක load වුණාට පස්සේ, JavaScript මඟින් වෙනස් කරන content වලටයි. උදාහරණ:

- Button එකක් click කරාම menu එකක් open වීම.
- Message එකක් "Saving..." වල ඉඳන් "Saved successfully." වලට වෙනස් වීම.
- User වෙන tab එකක් click කරද්දී tab panel එක වෙනස් වීම.
- Validation එකකට පස්සේ form error එකක් screen එකේ පේන්න ගැනීම.
- Dropdown එකක් දිගහැරීම.
- Notification එකක් screen එකට ඒම.

සරලවම කිව්වොත්, user, page එක පාවිච්චි කර කර ඉන්න අතරතුර JavaScript වලින් කරන හැම වෙනස්කමක්ම dynamic content ගණයට වැටෙනවා.

### Why Dynamic Content Needs Accessibility

**Concept Explanation (Sinhala):** JavaScript වලින් නිහඬවම (silently) යම් දෙයක් වෙනස් කළොත්, සමහර users ලට ඒ වෙනස්කම මඟහැරෙන්න පුළුවන්.

**දුර්වල ක්‍රමය (Weak HTML):**

```
<p id="status"></p>
```

```
status.textContent = "Saved successfully.";
```

**වඩා හොඳ HTML (Better HTML):**

```
<p id="status" aria-live="polite"></p>
```

**Code Breakdown (Sinhala):** දුර්වල ක්‍රමයේදී ඇස් පෙනෙන කෙනෙකුට text එක screen එකට ආපු බව පේනවා. හැබැයි screen reader එකක් පාවිච්චි කරන කෙනෙකුට මේක ශබ්ද නඟලා කියවෙන්නේ නැහැ, මොකද අපි මේ element එක live region එකක් විදිහට සකස් කරලා නැති නිසා. වඩා හොඳ HTML එකේදී paragraph එකට `aria-live="polite"` දීලා තියෙනවා. දැන් JavaScript වලින් status text එක වෙනස් කරපු සැනින්, assistive technology එකට පුළුවන් ඒ අලුත් වෙනස්කම user ට ශබ්ද නඟලා කියවන්න (announce කරන්න).

## Part D: Native HTML First (පළමුව ස්වදේශීය HTML භාවිතය)

### The Native-First Principle

**Concept Explanation (Sinhala):** "Native HTML first" කියන්නේ HTML වල යම් element එකක් හදලා තියෙන මුල්ම අරමුණට අනුව, ඒ නියම element එකම පාවිච්චි කිරීමයි.

**හොඳ උදාහරණ (Good examples):**

```
<button>Open menu</button>
<input required>
<select></select>
<a href="/profile">Profile</a>
```

**දුර්වල උදාහරණ (Weak examples):**

```
<div role="button">Open menu</div>
<span onclick="submitForm()">Submit</span>
```

මේ දුර්වල උදාහරණ වල තියෙන elements visually click කරන්න පුළුවන් වගේ පෙනුනත්, ඒවා සැබෑ buttons හෝ links වගේ ක්‍රියා කරන්නේ නැහැ.

### Why Native HTML Is Better

**Concept Explanation (Sinhala):** Native HTML elements වල උපතින්ම (built-in) accessibility features ගොඩක් ඇතුළත් වෙලා තියෙනවා.

සැබෑ `<button>` එකක් ගත්තොත්:

- Keyboard එකෙන් Tab key එක ඔබලා ඒකට focus වෙන්න පුළුවන්.
- Enter හෝ Space key එක ඔබලා ඒක active කරන්න පුළුවන්.
- ඒක button එකක් කියන එක screen reader එකට ඉබේම තේරෙනවා.
- Forms සහ browser behavior එක්ක ඒක තිබිම කරදරයක් නැතුව වැඩ කරනවා.

නමුත් `<div>` එකකට මේ කිසිම දෙයක් ඉබේ ලැබෙන්නේ නැහැ. ඔයා `<div role="button">` එකක් පාවිච්චි කරනවා නම්, ඔයාට සිද්ධ වෙනවා මේ හැමදේම manual (අතින්) කරන්න:

- Keyboard focus එක හැදීම.
- Enter/Space වලින් active කරන්න keyboard listeners ලියන්න.
- හරියටම role එක සහ state එක දාන්න.
- Disabled behavior එක වෙනම හදන්න.

**ආරම්භක රීතිය (Beginner rule):** හැමවෙලේම පළමුව Native HTML පාවිච්චි කරන්න. ARIA පාවිච්චි කරන්න ඕනේ native HTML වලින් ඒ custom behavior එක හරියටම විස්තර කරන්න බැරි වුණොත් විතරයි.

### ARIA Does Not Replace Good HTML

**Concept Explanation (Sinhala):** ARIA කියන්නේ Accessible Rich Internet Applications කියන එකයි. ARIA මඟින් custom හෝ dynamic components වලට අමතර තේරුමක් (extra meaning) එකතු කරන්න පුළුවන්. හැබැයි වැරදි HTML තේරීම වහන්න ARIA පාවිච්චි කරන්න එපා.

**නරක පුරුද්ද (Bad habit):**

```
<div role="button">Save</div>
```

**වඩා හොඳයි (Better):**

```
<button>Save</button>
```

**ආරම්භක රීතිය (Beginner rule):** ARIA පාවිච්චි කරන්න ඕනේ තේරුම තවත් වැඩි කරන්න (improve meaning) මිසක්, නිවැරදි HTML element එක වෙනුවට ආදේශකයක් විදිහට නෙමෙයි.

## Part E: ARIA Basics (ARIA හි මූලික කරුණු)

### What Is ARIA?

**Concept Explanation (Sinhala):** ARIA කියන්නේ assistive technology වලට element එකක් ගැන අමතර තොරතුරු සපයන attributes සමූහයක්.

**HTML Example:**

```
<button aria-expanded="false">Menu</button>
<p aria-live="polite">Saved successfully.</p>
<button role="tab" aria-selected="true">Overview</button>
```

සරලවම කිව්වොත්, ARIA මඟින් component එකක state (තත්ත්වය), relationships (සම්බන්ධතා), සහ behavior (හැසිරීම) screen reader එකට පැහැදිලි කරලා දෙනවා.

### ARIA Values Are Usually Strings

**Concept Explanation (Sinhala):** HTML වල ලියද්දී ARIA values හැමවෙලේම ලියන්නේ strings විදිහටයි. උදාහරණ: `aria-expanded="true"` හෝ `aria-expanded="false"`

JavaScript වලදී `getAttribute()` වලින් මේවා කියවද්දී ලැබෙන්නෙත් string එකක්මයි.

**Code Block:**

```
const isExpanded = button.getAttribute("aria-expanded") === "true";
button.setAttribute("aria-expanded", String(!isExpanded));
```

**Code Breakdown (Sinhala):**

- `button.getAttribute("aria-expanded")` එකෙන් ලැබෙන්නේ `"true"` හෝ `"false"` කියන string එක මිසක් JavaScript වල තියෙන boolean `true` හෝ `false` නෙමෙයි.
- ඒ නිසා `=== "true"` මඟින් අපි කියාගන්නේ string එකක් සමඟ සංසන්දනය කිරීමක්.
- ARIA attribute එකක් update කරද්දී boolean අගය නැවත string එකක් බවට convert කරන්න `String(!isExpanded)` කේතය පාවිච්චි වෙනවා.

## Part F: aria-expanded (දිගහැරුණු තත්ත්වය)

### What Is aria-expanded?

**Concept Explanation (Sinhala):** `aria-expanded` මඟින් screen reader එකට කියනවා යම් control එකක් (උදාහරණයක් විදිහට button එකක්) මඟින් තවත් element එකක් open හෝ close කරනවාද කියන එක.

පොදුවේ පාවිච්චි වන තැන්: dropdown menus, accordions, expandable sections, disclosure buttons, සහ navigation menus.

- `aria-expanded="false"` : මේ button එකෙන් පාලනය වන කොටස දැනට වැහිලයි (collapsed) තියෙන්නේ.
- `aria-expanded="true"` : පාලනය වන කොටස ඇරලයි (expanded) තියෙන්නේ.

### Basic aria-expanded Example

**Concept Explanation (Sinhala):** අපි සරල expandable menu එකක් හදලා බලමු. මේකේ HTML සහ JavaScript එකට ක්‍රියා කරන හැටි පහතින් බලාගන්න පුළුවන්.

**Code Block:**

```
<!-- HTML -->
<button id="menuButton" aria-expanded="false" aria-controls="menuList">
  Menu
</button>

<ul id="menuList" hidden>
  <li>Profile</li>
  <li>Settings</li>
  <li>Logout</li>
</ul>
```

```
// JavaScript
const button = document.getElementById("menuButton");
const menu = document.getElementById("menuList");

button.addEventListener("click", () => {
  const isExpanded = button.getAttribute("aria-expanded") === "true";
  button.setAttribute("aria-expanded", String(!isExpanded));
  menu.hidden = isExpanded;
});
```

**Code Breakdown (Sinhala):**

1. `const isExpanded = button.getAttribute("aria-expanded") === "true";`
   - මේ line එකෙන් button එකේ දැනට තියෙන ARIA state එක කියවනවා.
   - දැනට `aria-expanded="true"` නම් `isExpanded` එක `true` වෙනවා. දැනට `aria-expanded="false"` නම් `isExpanded` එක `false` වෙනවා.
2. `button.setAttribute("aria-expanded", String(!isExpanded));`
   - මෙකෙන් ARIA state එක flip (ප්‍රතිවිරුද්ධ අගයට හරවනවා) කරනවා.
   - `isExpanded` එක `true` වුණොත් ඒක `false` කරනවා, `false` වුණොත් `true` කරනවා.
3. `menu.hidden = isExpanded;`
   - මේකෙන් visual state එක (පේනවා/නොපේන එක) වෙනස් කරනවා.
   - Menu එක කලින් open වෙලා තිබුණා නම් (expanded), දැන් ඒක hide කරනවා (`menu.hidden = true`).
   - Menu එක කලින් වැහිලා තිබුණා නම් (collapsed), දැන් ඒක පෙන්නනවා (`menu.hidden = false`).

**ආරම්භක රීතිය (Beginner rule):** `aria-expanded` අගය සහ ඇත්තටම UI එකේ පේන්න තියෙන open/closed තත්ත්වය හැම තිස්සෙම එකිනෙකට ගැලපෙන්න (match වෙන්න) ඕනේ.

## Part G: aria-controls (පාලනය වන කොටස සම්බන්ධ කිරීම)

### What Is aria-controls?

**Concept Explanation (Sinhala):** `aria-controls` මඟින් button එක සහ ඒකෙන් පාලනය වෙන element එක අතර තියෙන සම්බන්ධතාවය screen reader එකට විස්තර කරනවා.

**HTML Example:**

```
<button aria-controls="menuList">Menu</button>
<ul id="menuList"></ul>
```

**aria-controls මඟින් ක්‍රියාකාරීත්වයක් (behavior) ඇති කරන්නේ නැහැ:** මේක වැදගත් කරුණක්. `aria-controls="menuList"` කියලා දැම්මා කියලා menu එක ඉබේ ඇරෙන්නේ හෝ වැහෙන්නේ නැහැ. Click behavior එක ඇති වෙන්නේ නැහැ. ඒකෙන් කරන්නේ සම්බන්ධතාවය විස්තර කිරීම විතරයි. JavaScript වලින් තමයි ක්‍රියාව සිදු කරන්න ඕනේ.

**ආරම්භක රීතිය (Beginner rule):** `aria-controls` වලින් කරන්නේ පාලනය කරන්නේ කුමන element එකද කියා පෙන්වීමයි. සැබෑ පාලනය (actual controlling) කරන්නේ JavaScript මඟින්.

## Part H: aria-owns (හිමිකාරීත්වය දැක්වීම)

### What Is aria-owns?

**Concept Explanation (Sinhala):** `aria-owns` මඟින් screen reader එකට කියනවා visual layout එකේ වෙන තැනක තිබුණත්, යම් element එකක් තවත් element එකක අයිතිකාරයා (parent/owner) විදිහට සලකන්න කියලා.

**HTML Example:**

```
<div role="listbox" aria-owns="option1 option2"></div>

<div id="option1" role="option">Small</div>
<div id="option2" role="option">Large</div>
```

**aria-owns පාවිච්චි කරන්නේ කවදාද?** සමහර වෙලාවට අපේ visual layout එක සහ HTML DOM structure එක එකිනෙකට ගැලපෙන්නේ නැහැ (උදාහරණයක් විදිහට, popup styles නිසා DOM එකේ වෙනමම තැනක code එක ලියවෙන්න වුණාම). අන්න ඒ වගේ වෙලාවට screen reader එකට logical relationship එක තේරුම් කරන්න `aria-owns` පාවිච්චි කරන්න පුළුවන්.

**ආරම්භක අනතුරු ඇඟවීම (Beginner Warning):** `aria-owns` කියන්නේ ටිකක් advanced attribute එකක්. සාමාන්‍ය beginner projects වලට මේක ඕන වෙන්නේ නැහැ. පුළුවන් හැමවෙලේම සරල DOM structure එකක් පාවිච්චි කරන්න:

```
<div>
  <button>Choose size</button>
  <ul>
    <li>Small</li>
    <li>Large</li>
  </ul>
</div>
```

**ආරම්භක රීතිය (Beginner rule):** හැකි සෑම විටම සම්බන්ධිත elements, DOM එකේ එක ළඟ තබා ගන්න. `aria-owns` පාවිච්චි කරන්න ඕනේ විශේෂිත custom components වලදී පමණි.

## Part I: hidden (මඟ හැරීම / සැඟවීම)

### What Is hidden?

**Concept Explanation (Sinhala):** `hidden` කියන්නේ element එකක් සඟවන්න පාවිච්චි කරන HTML attribute එකක්.

**HTML & JS Example:**

```
<ul id="menuList" hidden>
  <li>Profile</li>
</ul>
```

```
menu.hidden = true;  // Hide කරයි
menu.hidden = false; // Show කරයි
```

### hidden and aria-expanded Together

**Concept Explanation (Sinhala):** අපි expandable content එකක් හදද්දී මේ දෙකම එකට update කරන්න ඕනේ:

**Open කරද්දී:**

```
button.setAttribute("aria-expanded", "true");
menu.hidden = false;
```

**Close කරද්දී:**

```
button.setAttribute("aria-expanded", "false");
menu.hidden = true;
```

**ආරම්භක රීතිය (Beginner rule):** `aria-expanded` එක එකක් කියද්දී, `hidden` attribute එක තව එකක් කරන්න ඉඩ දෙන්න එපා. හැමවෙලේම මේ දෙක එකට ගැලපෙන්න ඕනේ.

## Part J: aria-live (සජීවී කලාප)

### What Is aria-live?

**Concept Explanation (Sinhala):** `aria-live` මඟින් screen reader එකට කියනවා යම් element එකක් ඇතුළේ තියෙන content එක dynamic විදිහට වෙනස් වෙන්න පුළුවන් බවත්, එහෙම වෙනස් වුණොත් ඒක user ට ශබ්ද නඟලා කියවන්න (announce කරන්න) ඕන බවත්.

පොදුවේ පාවිච්චි වන අවස්ථා: save messages, loading messages, form validation messages, cart update messages, search result counts, සහ notifications.

### aria-live="polite" vs aria-live="assertive"

**Concept Explanation (Sinhala):** `aria-live` වලට ප්‍රධාන වශයෙන් දාන්න පුළුවන් values දෙකක් තියෙනවා:

1. `aria-live="polite"`
   - තේරුම: "මේක සුහදශීලීව (polite) කියවන්න."
   - Screen reader එක දැනට වෙන දෙයක් කියමින් ඉන්නවා නම්, ඒක ඉවර වුණාම සුදුසු වෙලාවක මේ අලුත් message එක කියවනවා. සාමාන්‍ය status messages වලට මේක පාවිච්චි කරන්න.
   - උදාහරණ: `<p aria-live="polite">Saved successfully.</p>`
2. `aria-live="assertive"`
   - තේරුම: "මේක දැන්ම, හදිසියෙන් (assertive) කියවන්න!"
   - Screen reader එක දැනට කියව ඉන්න දේ මඟින් නවත්වලා, වහාම මේ අලුත් message එක කියවනවා. ඉතාම හදිසි සහ වැදගත් පණිවිඩ වලට විතරක් මේක පාවිච්චි කරන්න.
   - උදාහරණ: session timeout warnings හෝ critical errors.

**ආරම්භක අනතුරු ඇඟවීම (Beginner warning):** සාමාන්‍ය updates වලට `assertive` පාවිච්චි කරන්න එපා. ඒකෙන් user කර කර ඉන්න වැඩේට බාධා වෙන්න පුළුවන්.

### Live Region Example

**Concept Explanation (Sinhala):** සරල live region එකක් ක්‍රියා කරන හැටි පහත කේතයෙන් බලාගන්න පුළුවන්.

**Code Block:**

```
<!-- HTML -->
<p id="status" aria-live="polite"></p>
<button id="saveButton">Save</button>
```

```
// JavaScript
const status = document.getElementById("status");
const saveButton = document.getElementById("saveButton");

saveButton.addEventListener("click", () => {
  status.textContent = "Saved successfully.";
});
```

**Code Breakdown (Sinhala):**

1. User "Save" button එක click කරනවා.
2. JavaScript මඟින් `#status` element එකේ text එක `"Saved successfully."` වලට වෙනස් කරනවා.
3. Element එකට `aria-live="polite"` දීලා තියෙන නිසා screen reader එක මේ වෙනස්කම හඳුනාගෙන, user ට ඒක ශබ්ද නඟලා කියවනවා.

**ආරම්භක රීතිය (Beginner rule):** JavaScript වලින් page එක load වුණාට පස්සේ වැදගත් පණිවිඩයක් වෙනස් කරනවා නම්, හැමවෙලේම `aria-live` පාවිච්චි කරන්න.

## Part K: Common ARIA States for Custom Controls (Custom Controls සඳහා පොදු ARIA States)

### What Is a Custom Control?

**Concept Explanation (Sinhala):** Custom control එකක් කියන්නේ normal HTML elements පාවිච්චි කරන්නේ නැතුව, අපේම අවශ්‍යතාවයකට div/span/button එකතු කරලා අපිම හදන interactive component එකකටයි (උදා: custom tab interface, custom dropdown, custom switch, custom checkbox, listbox).

**ආරම්භක රීතිය (Beginner rule):** Custom control එකක් හදද්දී, ඒකේ role, state, keyboard behavior, සහ visual behavior එක පැහැදිලිව assistive technology වලට සන්නිවේදනය කරන්න (communicate කරන්න) ඕනේ.

## Part L: aria-selected (තෝරාගත් තත්ත්වය)

### What Is aria-selected?

**Concept Explanation (Sinhala):** `aria-selected` මඟින් screen reader එකට කියනවා තියෙන elements කිහිපයකින් දැනට select වෙලා තියෙන්නේ මොන item එකද කියලා (විශේෂයෙන්ම tabs, listbox options වලදී).

**HTML Example:**

```
<button role="tab" aria-selected="true">Overview</button>
<button role="tab" aria-selected="false">Details</button>
```

### Updating aria-selected

**Concept Explanation (Sinhala):** Tab එකක් මාරු කරද්දී, JavaScript වලින් ඒ හැම tab එකකම states update වෙන්න ඕනේ:

**Code Block:**

```
// Active Tab එකට:
tab.setAttribute("aria-selected", "true");
tab.tabIndex = 0;
panel.hidden = false;

// Inactive Tab එකට:
tab.setAttribute("aria-selected", "false");
tab.tabIndex = -1;
panel.hidden = true;
```

**Code Breakdown (Sinhala):**

- Active වන tab එකේ `aria-selected` එක `true` කරලා, `tabIndex = 0` මඟින් keyboard focus එක දෙනවා. ඒ වගේම panel එකේ `hidden` එක `false` කරනවා.
- Inactive වන tab එකේ `aria-selected` එක `false` කරලා, `tabIndex = -1` මඟින් keyboard focus එක මඟහරිනවා. ඒ වගේම panel එකේ `hidden` එක `true` කරනවා.

## Part M: aria-disabled (අක්‍රිය තත්ත්වය)

### What Is aria-disabled?

**Concept Explanation (Sinhala):** `aria-disabled` මඟින් screen reader එකට කියනවා යම් custom control එකක් දැනට අක්‍රිය (disabled) වෙලයි තියෙන්නේ කියලා.

**HTML Example:**

```
<button id="submitButton" aria-disabled="true">Submit</button>
```

### Important: aria-disabled Does Not Stop Clicks

**Concept Explanation (Sinhala):** මේක ඉතාම වැදගත් කරුණක්! HTML වල `disabled` attribute එක දැම්මම button එක click කරන්න බැරි වුණාට, ARIA attribute එකක් වන `aria-disabled="true"` දැම්මා කියලා JavaScript වලින් වෙන click behavior එක ඉබේ නවතින්නේ නැහැ. ඒක නවත්වන්න ඔයාම JavaScript කේතය ලියන්න ඕනේ.

**Code Block:**

```
const submitButton = document.getElementById("submitButton");

submitButton.addEventListener("click", () => {
  const isDisabled = submitButton.getAttribute("aria-disabled") ===
"true";

  if (isDisabled) {
    return; // Button එක disabled නම්, මෙතනින් පහළ code එක run වෙන්නේ
නැහැ.
  }

  console.log("Submitted");
});
```

**Code Breakdown (Sinhala):**

1. `const isDisabled = submitButton.getAttribute("aria-disabled") === "true"` මඟින් button එක දැනට disabled state එක check කරනවා.
2. `if (isDisabled) { return; }` මඟින් අගය `true` නම් function එක එතනින්ම නවත්වනවා.

**Native Disabled Is Usually Better:** සැබෑ buttons සඳහා හැම තිස්සෙම HTML වල එන native `disabled` attribute එක පාවිච්චි කරන්න:

```
<button disabled>Submit</button>
```

`aria-disabled` පාවිච්චි කරන්නේ custom controls වලදී පමණි.

## Part N: aria-haspopup (පොප්-අප් ඇති බව දැක්වීම)

### What Is aria-haspopup?

**Concept Explanation (Sinhala):** `aria-haspopup` මඟින් screen reader එකට කියනවා මේ control එක click කරාම popup එකක් (menu, listbox, dialog වගේ කොටසක්) open වෙනවා කියලා.

**HTML Example:**

```
<button aria-haspopup="menu" aria-expanded="false">Options</button>
```

### Common aria-haspopup Values

**Concept Explanation (Sinhala):**

- `"true"` : මොකක් හරි popup එකක් open වෙනවා කියලා පෙන්වයි.
- `"menu"` : Menu එකක් open වෙනවා කියලා පෙන්වයි.
- `"listbox"` : තෝරාගත හැකි ලැයිස්තුවක් (selectable list) open වෙනවා කියලා පෙන්වයි.
- `"dialog"` : Dialog box එකක් open වෙනවා කියලා පෙන්වයි.

**aria-haspopup මඟින් කිසිවක් open කරන්නේ නැහැ:** අනෙකුත් ARIA attributes වගේම, `aria-haspopup` වලින් කරන්නේ විස්තර කිරීමක් විතරයි. ඒක ඇරගන්න JavaScript ලියන්නම ඕනේ.

## Part O: aria-required (අනිවාර්යය බව දැක්වීම)

### What Is aria-required?

**Concept Explanation (Sinhala):** `aria-required` මඟින් screen reader එකට කියනවා යම් field එකක් අනිවාර්යයෙන්ම පිරවිය යුතුයි (required) කියලා.

**HTML Example:**

```
<input aria-required="true">
```

**Native required Is Usually Better:** සාමාන්‍ය input fields වලට HTML වල තියෙන native `required` attribute එක පාවිච්චි කරන එක වඩා හොඳයි:

```
<input required>
```

`aria-required` පාවිච්චි කළ යුත්තේ native `required` සපෝර්ට් නොකරන custom form-like controls වලදී පමණි.

## Part P: aria-checked (තෝරාගත් තත්ත්වය - Checkbox / Switches)

### What Is aria-checked?

**Concept Explanation (Sinhala):** `aria-checked` මඟින් screen reader එකට කියනවා custom checkbox, radio button, හෝ switch එකක් දැනට checked (තෝරාගෙන) ද නැද්ද කියලා.

**HTML Example:**

```
<button role="switch" aria-checked="false" id="darkModeSwitch">Dark
mode</button>
```

### Toggling aria-checked

**Concept Explanation (Sinhala):** JavaScript වලින් `aria-checked` අගය toggle කරන්නේ (මාරු කරන්නේ) මෙහෙමයි:

**Code Block:**

```
const switchButton = document.getElementById("darkModeSwitch");

switchButton.addEventListener("click", () => {
  const isChecked = switchButton.getAttribute("aria-checked") === "true";
  switchButton.setAttribute("aria-checked", String(!isChecked));
});
```

**Code Breakdown (Sinhala):**

1. `const isChecked = switchButton.getAttribute("aria-checked") === "true"` මඟින් Switch එක දැනට checked ද කියලා කියවනවා.
2. `switchButton.setAttribute("aria-checked", String(!isChecked))` මඟින් checked තත්ත්වය flip කරලා (ප්‍රතිවිරුද්ධ අගයට හරවලා) string එකක් විදිහට නැවත set කරනවා.

## Part Q: Keyboard Access (Keyboard එකෙන් ඇතුළු වීම)

### Why Keyboard Access Matters

**Concept Explanation (Sinhala):** හැම user කෙනෙක්ම mouse එකක් පාවිච්චි කරන්නේ නැහැ. සමහර අය මුළු web site එකම පාවිච්චි කරන්නේ keyboard එකෙන් විතරයි. ඔවුන් පාවිච්චි කරන keys:

- **Tab** සහ **Shift + Tab** (elements අතර මාරු වීමට)
- **Enter** සහ **Space** (buttons activate කිරීමට)
- **Arrow keys** (මෙනු හෝ ටැබ් අතර යාමට)
- **Home** සහ **End** (මුලට හෝ අගට යාමට)
- **Escape** (පොප්-අප් හෝ මෙනු වසා දැමීමට)

### Native Buttons Already Support Keyboard Use

**Concept Explanation (Sinhala):** සැබෑ `<button>` එකක් උපතින්ම keyboard එකට සපෝට් කරනවා:

- User ට **Tab** එකෙන් ඒකට focus වෙන්න පුළුවන්.
- **Enter** හෝ **Space** වලින් ඒක click කරන්න පුළුවන්.

හැබැයි `<div>` එකක් click කරවන්න හැදුවොත් ඒක keyboard එකෙන් කරන්න බැහැ. **දුර්වල ක්‍රමය (Weak):**

```
<div onclick="openMenu()">Open menu</div>
```

**වඩා හොඳයි (Better):**

```
<button id="menuButton">Open menu</button>
```

## Part R: Beginner Mental Model (ප්‍රශ්න 3ක මානසික ආකෘතිය)

**Concept Explanation (Sinhala):** ඔයා interactive component එකක් හදද්දී, මේ ප්‍රශ්න 3 තමන්ගෙන්ම අහන්න:

1. **Question 1: What Changes Visually? (ඇස් දෙකට පෙනෙන්නේ මොනවාද?)**
   - උදාහරණ: Menu එකක් පේන්න ගන්නවා, Tab panel එක වෙනස් වෙනවා, Status text එකක් වෙනස් වෙනවා.
2. **Question 2: What Changes Semantically? (ARIA/Semantic state එක වෙනස් වෙන්නේ කොහොමද?)**
   - උදාහරණ: `aria-expanded` හෝ `aria-selected` එක `"false"` සිට `"true"` වලට වෙනස් වීම.
3. **Question 3: Can the Keyboard Use It? (Keyboard එකෙන් මේක කරන්න පුළුවන්ද?)**
   - උදාහරණ: User ට **Tab** එකෙන් focus වෙන්න සහ **Enter** / **Space** වලින් activate කරන්න පුළුවන්ද?

## Part S: Mini-Build 1 - Accessible Dropdown (ප්‍රවේශ්‍ය dropdown මෙනුවක් සාදමු)

### Goal

**Concept Explanation (Sinhala):** අපි හදන්න යන්නේ accessible dropdown මෙනුවක්. මේකේ:

- Button එකෙන් menu එක open සහ close කරනවා.
- `aria-expanded` එක open/closed state එකට ගැලපෙනවා.
- `aria-controls` මඟින් button එක සහ menu එක සම්බන්ධ කරනවා.
- `hidden` attribute එකෙන් menu එක ඇත්තටම සඟවනවා සහ පෙන්වනවා.
- `aria-live` මඟින් වෙනස් වන තත්ත්වය ශබ්ද නඟලා පවසනවා (announce කරනවා).

**Code Block:**

```
<!-- HTML -->
<button id="menuButton" aria-expanded="false" aria-controls="menuList">
  Menu
</button>

<ul id="menuList" hidden>
  <li><a href="/profile">Profile</a></li>
  <li><a href="/settings">Settings</a></li>
  <li><a href="/logout">Logout</a></li>
</ul>

<p id="status" aria-live="polite"></p>
```

```
// JavaScript
const menuButton = document.getElementById("menuButton");
const menuList = document.getElementById("menuList");
const status = document.getElementById("status");

menuButton.addEventListener("click", () => {
  const isExpanded = menuButton.getAttribute("aria-expanded") === "true";
  const nextExpanded = !isExpanded;

  menuButton.setAttribute("aria-expanded", String(nextExpanded));
  menuList.hidden = !nextExpanded;

  if (nextExpanded) {
    status.textContent = "Menu opened.";
  } else {
    status.textContent = "Menu closed.";
  }
});
```

**Code Breakdown (Sinhala):**

- `menuButton.setAttribute("aria-expanded", String(nextExpanded))` මඟින් screen reader එකට button එක දිගහැරුණු බව කියයි.
- `menuList.hidden = !nextExpanded` මඟින් මෙනුව ඇත්තටම සඟවයි/පෙන්වයි.
- `status.textContent = "Menu opened."` මඟින් `aria-live` හරහා screen reader එක වෙනස්වීම ශබ්ද නඟා කියවයි.

## Part T: Mini-Build 2 — Accessible Status Message (ප්‍රවේශ්‍ය තත්ත්ව පණිවිඩයක් සාදමු)

### HTML & JavaScript Implementation

**Code Block:**

```
<!-- HTML -->
<button id="saveButton">Save</button>
<p id="saveStatus" aria-live="polite"></p>
```

```
// JavaScript
const saveButton = document.getElementById("saveButton");
const saveStatus = document.getElementById("saveStatus");

saveButton.addEventListener("click", () => {
  saveStatus.textContent = "Saved successfully.";
});
```

**Code Breakdown (Sinhala):**

- HTML එකේ status message එක දාන්න හදලා තියෙන paragraph (`<p>`) එකට `aria-live="polite"` දීලා තියෙනවා.
- Button එක click කරාම, JavaScript මඟින් ඒ paragraph එක ඇතුළට `"Saved successfully."` කියන text එක දානවා.
- ඒ සැනින් screen reader එක මේ වෙනස්කම හඳුනාගෙන, user ට ඒක ශබ්ද නඟලා කියවනවා.

## Part U: Mini-Build 3 - Accessible Tabs (ප්‍රවේශ්‍ය tabs අතුරුමුහුණතක් සාදමු)

### HTML & JavaScript Implementation

**Code Block:**

```
<!-- HTML -->
<div role="tablist" aria-label="Account sections">
  <button
    id="tab-overview"
    role="tab"
    aria-selected="true"
    aria-controls="panel-overview"
    tabindex="0"
  >
    Overview
  </button>

  <button
    id="tab-settings"
    role="tab"
    aria-selected="false"
    aria-controls="panel-settings"
    tabindex="-1"
  >
    Settings
  </button>
</div>

<section
  id="panel-overview"
  role="tabpanel"
  aria-labelledby="tab-overview"
>
  Overview content
</section>

<section
  id="panel-settings"
  role="tabpanel"
  aria-labelledby="tab-settings"
  hidden
>
  Settings content
</section>
```

```
// JavaScript
const overviewTab = document.getElementById("tab-overview");
const settingsTab = document.getElementById("tab-settings");
const overviewPanel = document.getElementById("panel-overview");
const settingsPanel = document.getElementById("panel-settings");

function selectOverview() {
  overviewTab.setAttribute("aria-selected", "true");
  overviewTab.tabIndex = 0;
  overviewPanel.hidden = false;

  settingsTab.setAttribute("aria-selected", "false");
  settingsTab.tabIndex = -1;
  settingsPanel.hidden = true;
}

function selectSettings() {
  settingsTab.setAttribute("aria-selected", "true");
  settingsTab.tabIndex = 0;
  settingsPanel.hidden = false;

  overviewTab.setAttribute("aria-selected", "false");
  overviewTab.tabIndex = -1;
  overviewPanel.hidden = true;
}

overviewTab.addEventListener("click", selectOverview);
settingsTab.addEventListener("click", selectSettings);
```

**Code Breakdown (Sinhala):**

- Active වන tab එකට `aria-selected` එක `true` දමා, `tabIndex` එක `0` කර, panel එකේ `hidden` එක `false` කරයි.
- Inactive වන tabs වලට `aria-selected` එක `false` දමා, `tabIndex` එක `-1` කර, panel එකේ `hidden` එක `true` කරයි.

## Part V: Common Sprint 14 Mistakes (පොදු වැරදි 6ක්)

**Concept Explanation (Sinhala):** අපි වැඩ කරද්දී බහුලවම වෙන වැරදි සහ ඒවා නිවැරදි කරගන්නේ කොහොමද කියලා බලමු:

- **Mistake 1: Only Changing CSS (CSS පමණක් වෙනස් කිරීම):**
  - වැරදි ක්‍රමය: `menu.classList.toggle("open");`
  - නිවැරදි ක්‍රමය: `const isOpen = menu.classList.toggle("open"); button.setAttribute("aria-expanded", String(isOpen)); menu.hidden = !isOpen;`
- **Mistake 2: aria-expanded Does Not Match the UI:**
  - වැරදි ක්‍රමය: UI එකේ මෙනුව පෙනෙන්න තිබුණත් `aria-expanded="false"` ලෙස තිබීම. හැමවෙලේම මෙනුව පෙනෙන විට `aria-expanded="true"` විය යුතුය.
- **Mistake 3: Using ARIA Instead of Native HTML:**
  - වැරදි ක්‍රමය: `<div role="button">Submit</div>`
  - නිවැරදි ක්‍රමය: `<button>Submit</button>`
- **Mistake 4: Thinking aria-disabled Disables the Button:**
  - වැරදි වැටහීම: `aria-disabled="true"` දැමූ පමණින් click ක්‍රියාව නතර නොවේ. JavaScript click event එක ඇතුළත `button.getAttribute("aria-disabled") === "true"` ද යන්න check කර `return` කළ යුතුය.
- **Mistake 5: Using aria-live="assertive" Too Much:**
  - වැරදි ක්‍රමය: සාමාන්‍ය alerts වලට `assertive` දැමීම. සාමාන්‍ය messages වලට හැමවිටම `polite` පාවිච්චි කරන්න.
- **Mistake 6: Creating Custom Controls Without Keyboard Support:**
  - වැරදි ක්‍රමය: Keyboard එකෙන් focus විය නොහැකි සහ Enter/Space වලින් ක්‍රියා නොකරන custom controls සෑදීම.

## Part W: Core Code Patterns to Memorize (කටපාඩම් කරගත යුතු ප්‍රධාන කේත රටා)

**Concept Explanation (Sinhala):** නිතරම ප්‍රායෝගිකව භාවිත වන කේත රටා (patterns) 5ක් මෙන්න:

1. **Toggle Expanded State:**

```
const isExpanded = button.getAttribute("aria-expanded") === "true";
button.setAttribute("aria-expanded", String(!isExpanded));
menu.hidden = isExpanded;
```

2. **Update a Live Region:**

```
<p id="status" aria-live="polite"></p>
```

```
status.textContent = "Saved successfully.";
```

3. **Block Disabled Custom Control Behavior:**

```
button.addEventListener("click", () => {
  const isDisabled = button.getAttribute("aria-disabled") === "true";
  if (isDisabled) {
    return;
  }
  runAction();
});
```

4. **Toggle Checked State:**

```
const isChecked = switchButton.getAttribute("aria-checked") === "true";
switchButton.setAttribute("aria-checked", String(!isChecked));
```

5. **Select a Tab:**

```
activeTab.setAttribute("aria-selected", "true");
activeTab.tabIndex = 0;
activePanel.hidden = false;

inactiveTab.setAttribute("aria-selected", "false");
inactiveTab.tabIndex = -1;
inactivePanel.hidden = true;
```

## Part X: Sprint 14 Debug Checklist (වැරදි සෙවීමේ පිරික්සුම් ලැයිස්තුව)

**Concept Explanation (Sinhala):** ඔයා හදපු interactive component එකක් හරියටම වැඩද කියලා check කරගන්න මේ checklist එක පාවිච්චි කරන්න:

- **Visual State:**
  - Component එක visually හරියටම open, close, select, disable, හෝ update වෙනවද?
  - පේන්න තියෙන විදිහ (visual state) සහ JavaScript state එක එකිනෙකට ගැලපෙනවද?
- **Accessibility State:**
  - `aria-expanded` එක open/closed state එකට ගැලපෙනවද?
  - `aria-selected` එක selected state එකට ගැලපෙනවද?
  - `aria-checked` එක checked state එකට ගැලපෙනවද?
  - `aria-disabled` එක disabled state එකට ගැලපෙනවද?
  - `aria-controls` එකෙන් නිවැරදි element ID එකටම point කරලා තියෙනවද?
  - වෙනස් වන වැදගත් status messages වලට `aria-live` දීලා තියෙනවද?
- **Keyboard Use:**
  - User ට **Tab** key එකෙන් control එකට ළඟා වෙන්න පුළුවන්ද?
  - Keyboard එකෙන් ඒක activate කරන්න පුළුවන්ද?
  - Custom tabs වල arrow keys වලින් ඒවා අතර මාරු වෙන්න පුළුවන්ද?
- **Native HTML:**
  - සැබෑ `<button>`, `<input>`, `<select>` හෝ `<a href="...">` පාවිච්චි කරන්න පුළුවන් හැම තැනකදීම ඒවා මුලින්ම පාවිච්චි කරලා තියෙනවද?

## Part Y: Concept Revision Questions (පුනරීක්ෂණ ප්‍රශ්න)

**Concept Explanation (Sinhala):** මෙන්න ඔයාගේ න්‍යායාත්මක දැනුම (theory knowledge) මැනගන්න ප්‍රශ්න සහ පිළිතුරු කිහිපයක්:

1. **What is the difference between visual state and accessibility state?**
   - පිළිතුර: Visual state කියන්නේ සාමාන්‍ය user කෙනෙකුට ඇස් දෙකෙන් පේන්න තියෙන දේ (උදා: මෙනුවක් ඇරී තිබීම). Accessibility state කියන්නේ screen readers වගේ උපකාරක මෙවලම් වලට කේතය (HTML/ARIA) මඟින් තේරුම් ගන්න තත්ත්වය (උදා: `aria-expanded="true"` වීම).
2. **Why is changing CSS alone not enough for accessible dynamic components?**
   - පිළිතුර: CSS වෙනස් කරාම වෙනස් වෙන්නේ පෙනුම විතරයි (visual state). Screen reader එකකට ඒ වෙනස තේරෙන්නේ නැහැ ARIA attributes හෝ HTML states වෙනස් කරලා නැත්නම්.
3. **What does aria-expanded communicate?**
   - පිළිතුර: Button එකකින් පාලනය වෙන content එකක් දැනට ඇරලද (expanded) නැත්නම් වැහිලද (collapsed) තියෙන්නේ කියන එක.
4. **What does aria-controls communicate?**
   - පිළිතුර: යම් control එකක් මඟින් පාලනය කරන්නේ කුමන ID එකක් සහිත element එකක්ද කියන එක (සම්බන්ධතාවය විස්තර කරයි).
5. **Does aria-controls create behavior by itself?**
   - පිළිතුර: නැහැ, ඒකෙන් සම්බන්ධතාවය විස්තර කරනවා පමණයි. ක්‍රියාකාරීත්වය හදන්න JavaScript ඕනේ.
6. **What does hidden do?**
   - පිළිතුර: Element එකක් visually සහ screen readers වලින් සඟවනවා (hide කරයි).
7. **What is aria-live used for?**
   - පිළිතුර: Page එක load වුණාට පස්සේ වෙනස් වෙන වැදගත් status messages, screen reader එකෙන් ශබ්ද නඟලා කියවන්න (announce කරන්න).
8. **What is the difference between aria-live="polite" and aria-live="assertive"?**
   - පිළිතුර: `polite` එකෙන් සුදුසු වෙලාවක බාධාවකින් තොරව message එක කියවනවා. `assertive` එකෙන් user කරන වැඩේ නවත්තලා, වහාම message එක කියවනවා.
9. **Why should assertive be used carefully?**
   - පිළිතුර: නිතරම user ට බාධා කිරීමෙන් වෙබ් අඩවිය පාවිච්චි කිරීම අපහසු වන නිසා.
10. **What does aria-selected communicate?**
    - පිළිතුර: Tabs හෝ listbox වගේ තැන් වල දැනට select වෙලා තියෙන item එක මොකක්ද කියන එක.
11. **What does aria-disabled communicate?**
    - පිළිතුර: Custom control එකක් දැනට අක්‍රිය (disabled) වෙලයි තියෙන්නේ කියන එක.
12. **Does aria-disabled automatically stop click behavior?**
    - පිළිතුර: නැත, click වීම වැළැක්වීමට JavaScript කේතය අපිම ලිවිය යුතුය.
13. **What does aria-checked communicate?**
    - පිළිතුර: Custom checkbox, switch, හෝ radio button එකක් checked වෙලාද නැද්ද කියන එක.
14. **What does aria-haspopup communicate?**
    - පිළිතුර: මේ button එක click කරාම menu එකක් හෝ dialog එකක් වගේ popup එකක් මතු වෙන බව.
15. **What does aria-required communicate?**
    - පිළිතුර: Custom input field එකක් අනිවාර්යයෙන්ම පිරවිය යුතු බව.
16. **Why is native required usually better than aria-required on normal inputs?**
    - පිළිතුර: Native `required` attribute එකෙන් browser එකේ built-in form validation එක සමඟ ඉබේම වැඩ කරන නිසා.
17. **Why is a real `<button>` usually better than `<div role="button">`?**
    - පිළිතුර: Real button එකකට keyboard focus, Enter/Space activation වගේ දේවල් උපතින්ම (built-in) ලැබෙන නිසා.
18. **Why do custom controls need keyboard support?**
    - පිළිතුර: Mouse පාවිච්චි නොකරන, keyboard එකෙන් පමණක් වෙබ් අඩවි පාවිච්චි කරන users ලට ඒවා ක්‍රියා කරවීම සඳහා.
19. **What three questions should you ask for every interactive component?**
    - පිළිතුර: 1. What changes visually? 2. What changes semantically? 3. Can the keyboard use it?
20. **What does it mean to keep visual state and assistive-technology state in sync?**
    - පිළිතුර: User ට ඇස් දෙකට පේන වෙනස්කම (visual state), screen reader එකටත් කේතය හරහා එක වගේම තේරුම් ගැනීමට සැලැස්වීම (accessibility state).

## Part Z: Sprint 14 Passing Standard (සමත් වීමේ ප්‍රමිතිය)

**Concept Explanation (Sinhala):** ඔයා Sprint 14 සාර්ථකව සමත් වෙන්න නම්, පහත තියෙන JavaScript රටාව මතකයෙන් ලියන්න සහ පැහැදිලි කරන්න පුළුවන් වෙන්න ඕනේ:

**Code Block:**

```
const expanded = button.getAttribute("aria-expanded") === "true";
button.setAttribute("aria-expanded", String(!expanded));
menu.hidden = expanded;
```

**පැහැදිලි කිරීම (Explanation):**

- `getAttribute()` මඟින් දැනට තියෙන ARIA state එක කියවයි.
- HTML attributes වලින් ලැබෙන ARIA values හැමවෙලේම strings වේ.
- `aria-expanded` අගය, controlled content එක ඇත්තටම open/closed ද යන්න මත තීරණය වේ.
- `hidden` attribute එක මඟින් visual පෙනුම පාලනය කරයි.
- JavaScript මඟින් හැම තිස්සෙම visual state සහ accessibility state එකට update කළ යුතුය.
- `aria-controls` මඟින් elements අතර සම්බන්ධය පෙන්වුවත්, එයින් behavior එකක් නිර්මාණය නොවේ.
- `aria-live` පාවිච්චි කරන්නේ dynamic status messages වලටයි.
- `aria-disabled` මඟින් disabled තත්ත්වය විස්තර කරනවා පමණි, behavior එක නතර නොකරයි.
- `aria-selected` මඟින් tabs/listbox වල දැනට තෝරාගත් element එක පෙන්වයි.
- `aria-checked` මඟින් custom checkboxes/switches වල checked තත්ත්වය පෙන්වයි.
- `aria-haspopup` මඟින් popup එකක් open වන බව කියයි.
- `aria-required` මඟින් custom field එකක් අනිවාර්යයෙන්ම පිරවිය යුතු බව කියයි.
- ගැටලුව විසඳීමට හැකි සෑම විටම custom controls වලට වඩා Native HTML elements භාවිතය වඩාත් සුදුසු වේ.
- Keyboard පාවිච්චි කරන අයට හැම interactive control එකක්ම පාවිච්චි කළ හැකි විය යුතුය.

**Sprint 14 අවසාන සාරාංශය (Final Summary):** Sprint 14 වලින් ඔයාට කියා දෙන්නේ JavaScript interactions accessible කරන්නේ කොහොමද කියන එකයි. ඒ සඳහා:

1. User ඇස් දෙකෙන් දකින දේ (Visual state)
2. Assistive technology වලට තේරෙන දේ (Accessibility state)
3. Keyboard එකෙන් operate කරන්න පුළුවන් දේ (Keyboard behavior) හැමවෙලේම එකිනෙකට sync එකේ (සමගාමීව) තබා ගත යුතුය.